Assignment 12.4

Name: Zobiya Fatima

Batch No: 14

Hall Ticket No: 2303A51879

Task 1: Bubble Sort for Ranking Exam Scores

Scenario: You are working on a college result processing system where a small list of student scores needs to be sorted after every internal assessment.

Task Description

• Implement Bubble Sort in Python to sort a list of student scores.

• Use an AI tool to:

o Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes

o Identify early-termination conditions when the list becomes sorted

o Provide a brief time complexity analysis

Expected Outcome

• A Bubble Sort implementation with:

o AI-generated comments explaining the logic

o Clear explanation of best, average, and worst-case complexity

o Sample input/output showing sorted scores

```python
def bubble_sort(scores):
    n = len(scores)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if scores[j] > scores[j + 1]:
                scores[j], scores[j + 1] = scores[j + 1], scores[j]
                swapped = True
        if not swapped:
            print(f"Early termination on pass {i + 1} (List already sorted)")
            break
    return scores
# Sample Input
scores = [78, 92, 85, 60, 88, 75]
print("Original Scores:", scores)
sorted_scores = bubble_sort(scores)
print("Sorted Scores:", sorted_scores)
```

Output:

```
Original Scores: [78, 92, 85, 60, 88, 75]
Early termination on pass 5 (List already sorted)
Sorted Scores: [60, 75, 78, 85, 88, 92]
```

Justification for Using Bubble Sort

Bubble Sort is simple and easy to implement, making it suitable for small datasets like internal assessment scores.
It repeatedly compares adjacent elements and swaps them if they are in the wrong order.
The algorithm includes an early termination optimization, which improves performance if the list becomes sorted early.
Since the number of students in internal exams is usually small, its $O(n^2)$ complexity is acceptable.

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Scenario: You are maintaining an attendance system where student roll numbers are already almost sorted, with only a few late updates.

Task Description

• Start with a Bubble Sort implementation.

• Ask AI to:

o Review the problem and suggest a more suitable sorting algorithm

o Generate an Insertion Sort implementation

o Explain why Insertion Sort performs better on nearly sorted data

• Compare execution behavior on nearly sorted input

Expected Outcome

• Two sorting implementations:

o Bubble Sort

o Insertion Sort

• AI-assisted explanation highlighting efficiency differences for

partially sorted datasets

Bubble Sort Implementation:

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False  # Tracks if any swap happens in this pass
        for j in range(0, n - i - 1):
            # Compare adjacent roll numbers
            if arr[j] > arr[j + 1]:
                # Swap if out of order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # Early termination if no swaps occur
        if not swapped:
            break
    return arr
# Nearly sorted attendance roll numbers
attendance = [101, 102, 103, 105, 104, 106, 107]
print("Original Attendance:", attendance)
print("Bubble Sort Result:", bubble_sort(attendance.copy()))
```

Output:

```
Original Attendance: [101, 102, 103, 105, 104, 106, 107]
Bubble Sort Result: [101, 102, 103, 104, 105, 106, 107]
```

Justification for Bubble Sort :

Bubble Sort is simple and easy to implement, making it suitable for small datasets.
It repeatedly compares adjacent elements and swaps them if they are out of order.
With the early termination optimization, it stops if no swaps occur in a pass.
However, it still performs multiple passes, making it less efficient for nearly sorted data.

Insertion Sort Implementation:

```python
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]  # Current roll number to insert
        j = i - 1
        # Shift elements that are greater than key
        # to one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        # Place key in correct sorted position
        arr[j + 1] = key
    return arr
# Nearly sorted attendance roll numbers
attendance = [101, 102, 103, 105, 104, 106, 107]
print("Original Attendance:", attendance)
print("Insertion Sort Result:", insertion_sort(attendance.copy()))
```

Output:

```
Original Attendance: [101, 102, 103, 105, 104, 106, 107]
Insertion Sort Result: [101, 102, 103, 104, 105, 106, 107]
```

Justification for Insertion Sort

Insertion Sort is highly efficient for nearly sorted data because it shifts only misplaced elements.
It builds the sorted portion of the list incrementally.
When the data is almost sorted, very few shifts are required, leading to faster execution.
Its best-case time complexity is O(n), making it more suitable for attendance records with minor updates.

Task 3: Searching Student Records in a Database

Scenario:You are developing a student information portal where users search for

student records by roll number.

Task Description

• Implement:

o Linear Search for unsorted student data

o Binary Search for sorted student data

• Use AI to:

o Add docstrings explaining parameters and return values

o Explain when Binary Search is applicable

o Highlight performance differences between the two searches

Expected Outcome

• Two working search implementations with docstrings

• AI-generated explanation of:

o Time complexity

o Use cases for Linear vs Binary Search

• A short student observation comparing results on sorted vs unsorted lists

Linear Search (For Unsorted Data):

```python
def linear_search(records, target_roll):
    for index, roll in enumerate(records):
        # Compare each element with target roll number
        if roll == target_roll:
            return index

    # If not found after checking all elements
    return -1
unsorted_records = [105, 101, 109, 103, 107]
sorted_records = sorted(unsorted_records)
target = 103
print("Linear Search (variable) unsorted_records: list[int]
      linear_search(unsorted_records, target))
```

Output:

```
Linear Search Result (Unsorted): 3
```

Justification for Linear Search:

Linear Search is suitable for unsorted student records because it does not require any specific order in the data.
It checks each element sequentially until the target roll number is found.
It is simple to implement and works well for small datasets.
However, its time complexity is O(n), making it slower for large databases.

Binary Search (For Sorted Data):

```python
def binary_search(records, target_roll):
    left = 0
    right = len(records) - 1
    while left <= right:
        mid = (left + right) // 2
        # Check if middle element is target
        if records[mid] == target_roll:
            return mid
        # If target is greater, search right half
        elif records[mid] < target_roll:
            left = mid + 1
        # If target is smaller, search left half
        else:
            right = mid - 1
    return -1
unsorted_records = [105, 101, 109, 103, 107]
sorted_records = sorted(unsorted_records)
target = 103
print("Binary Search Result (Sorted):",
      binary_search(sorted_records, target))
```

Output:

```
Binary Search Result (Sorted): 1
```

Justification for Binary Search

Binary Search is efficient for sorted student records because it repeatedly divides the search space in half.
It significantly reduces the number of comparisons compared to Linear Search.
Its time complexity is O(log n), making it highly efficient for large datasets.
However, it can only be applied when the data is sorted.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Scenario: You are part of a data analytics team that needs to sort large datasets received from different sources (random order, already sorted, and reverse sorted).

Task Description

• Provide AI with partially written recursive functions for:

o Quick Sort

o Merge Sort

• Ask AI to:

o Complete the recursive logic

o Add meaningful docstrings

o Explain how recursion works in each algorithm

• Test both algorithms on:

o Random data

o Sorted data

o Reverse-sorted data

Expected Outcome

• Fully functional Quick Sort and Merge Sort implementations

• AI-generated comparison covering:

o Best, average, and worst-case complexities

o Practical scenarios where one algorithm is preferred over the other

Quick Sort Implementation (Recursive):

```python
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[-1]
    # Partitioning step
    left = [x for x in arr[:-1] if x <= pivot]
    right = [x for x in arr[:-1] if x > pivot]
    # Recursive step:
    return quick_sort(left) + [pivot] + quick_sort(right)
random_data = [45, 12, 89, 33, 7, 56]
sorted_data = [7, 12, 33, 45, 56, 89]
reverse_data = [89, 56, 45, 33, 12, 7]

print("Quick Sort Results:")
print("Random:", quick_sort(random_data))
print("Sorted:", quick_sort(sorted_data))
print("Reverse:", quick_sort(reverse_data))
```

Output:

```
Quick Sort Results:
Random: [7, 12, 33, 45, 56, 89]
Sorted: [7, 12, 33, 45, 56, 89]
Reverse: [7, 12, 33, 45, 56, 89]
```

Justification for Quick Sort

Quick Sort is preferred for large, randomly ordered datasets because it has an average time complexity of **O(n log n)** and performs very fast in practice.
It works by selecting a pivot and partitioning the array into smaller subarrays recursively.
It requires less extra memory since it can be implemented in-place.
However, its worst-case time complexity is **O(n²)** when the data is already sorted or reverse sorted (with poor pivot selection).

Merge Sort Implementation (Recursive):

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]
    left_sorted = merge_sort(left_half)
    right_sorted = merge_sort(right_half)
    return merge(left_sorted, right_sorted)
def merge(left, right):
    sorted_list = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            sorted_list.append(left[i])
            i += 1
        else:
            sorted_list.append(right[j])
            j += 1
    sorted_list.extend(left[i:])
    sorted_list.extend(right[j:])
    return sorted_list
```

Output:

```
Merge Sort Results:
Random: [7, 12, 33, 45, 56, 89]
Sorted: [7, 12, 33, 45, 56, 89]
Reverse: [7, 12, 33, 45, 56, 89]
```

Justification for Merge Sort

Merge Sort guarantees a consistent time complexity of O(n log n) in all cases (best, average, and worst).
It uses a divide-and-conquer approach by splitting the array and merging sorted halves recursively.
It is stable, meaning it preserves the order of equal elements.
However, it requires additional memory O(n) for merging, making it less space-efficient than Quick Sort

Task 5: Optimizing a Duplicate Detection Algorithm

Scenario

You are building a data validation module that must detect duplicate

user IDs in a large dataset before importing it into a system.

Task Description

• Write a naive duplicate detection algorithm using nested loops.

• Use AI to:

o Analyze the time complexity

o Suggest an optimized approach using sets or dictionaries

o Rewrite the algorithm with improved efficiency

• Compare execution behavior conceptually for large input sizes

Expected Outcome

• Two versions of the algorithm:

o Brute-force ($O(n^2)$)

o Optimized (O(n))

• AI-assisted explanation showing how and why performance

Improved

Brute-Force Duplicate Detection ($O(n^2)$):

```python
def find_duplicates_bruteforce(user_ids):
    duplicates = []

    for i in range(len(user_ids)):
        for j in range(i + 1, len(user_ids)):
            # Compare every pair of elements
            if user_ids[i] == user_ids[j]:
                if user_ids[i] not in duplicates:
                    duplicates.append(user_ids[i])
    return duplicates
# Sample dataset with duplicate user IDs
user_ids = [101, 205, 309, 101, 450, 205, 678, 900, 309]

print("Original User IDs:")
print(user_ids)
# Brute-force method
duplicates_brute = find_duplicates_bruteforce(user_ids)
print("\nDuplicates (Brute Force):")
print(duplicates_brute)
```

Output :

```
Merge Sort Results:
Random: [7, 12, 33, 45, 56, 89]
Sorted: [7, 12, 33, 45, 56, 89]
Reverse: [7, 12, 33, 45, 56, 89]
```

Justification for Brute-Force Duplicate Detection

The brute-force method compares every element with every other element using nested loops.
It is simple and easy to implement, making it suitable for very small datasets.
However, it performs a large number of unnecessary comparisons as the dataset grows.
Its time complexity is $O(n^2)$, making it inefficient for large-scale data validation.

Optimized Duplicate Detection Using Set (O(n)):

```python
def find_duplicates_optimized(user_ids):
    seen = set()
    duplicates = set()

    for user_id in user_ids:
        # If already seen, it's a duplicate
        if user_id in seen:
            duplicates.add(user_id)
        else:
            seen.add(user_id)

    return list(duplicates)
# Sample dataset with duplicate user IDs
user_ids = [101, 205, 309, 101, 450, 205, 678, 900, 309]

print("Original User IDs:")
print(user_ids)
# Optimized method
duplicates_optimized = find_duplicates_optimized(user_ids)
print("\nDuplicates (Optimized using Set):")
print(duplicates_optimized)
```

Output:

```
Original User IDs:
[101, 205, 309, 101, 450, 205, 678, 900, 309]

Duplicates (Optimized using Set):
[205, 309, 101]
```

Justification for Optimized Duplicate Detection (Using Set)

The optimized method uses a set to track previously seen user IDs.
Set lookup operations take O(1) average time, eliminating repeated comparisons.
Each element is processed only once, reducing overall operations.
Its time complexity is O(n), making it highly efficient and scalable for large datasets.